

# Charge Time Prediction: Forecasting Electric Vehicle Battery's Charge Time Remaining Using Machine Learning

**Team members:** Alex Alvarez ([alexalva@berkeley.edu](mailto:alexalva@berkeley.edu)) , Alexandra Bauche ([alexandra.bauche@berkeley.edu](mailto:alexandra.bauche@berkeley.edu)), Maggie Royal ([mroyal7@berkeley.edu](mailto:mroyal7@berkeley.edu)), and Micah Collins ([micah.collins@berkeley.edu](mailto:micah.collins@berkeley.edu))

**Abstract:** Electric vehicle (EV) charging is more variable and time-consuming than refueling gas-powered vehicles, creating challenges for travel planning and fleet management. This project uses machine learning to improve charge time prediction using real-world data from Camber, a fleet charging management company. Traditional formulas oversimplify the problem, failing to account for nonlinear factors across different vehicles and chargers. We evaluated multiple models—including linear regression, random forest, XGBoost, RNNs with LSTM, GRU, TiDE, and TFT—and found that the GRU has the best results based on Mean Absolute Error (MAE) alongside being very efficient with a small parameter size. Our results demonstrate the potential of machine learning to enhance EV charging reliability and support broader EV adoption.

**Introduction:** Unlike conventional gas-power vehicles, which can be refueled quickly and predictably, EV charging can be prolonged as well as highly variable. This unpredictability is a significant concern for both consumers and institutions, particularly when planning travel or managing fleets of EV's. Understanding the factors that influence an EV's state of charge (SOC) is critical for efficient travel planning, charging management, and EV fleet administration. Machine learning, with its robust predictive modeling capabilities, presents an opportunity to alleviate these concerns by analyzing large volumes of EV charge time data from Camber, a company specialized in fleet charging solutions. We partnered with Camber to support their software, Camber Core, which allows for fleets to be monitored and managed in real-time. Leveraging machine learning models could allow for accurate predictions of the time allocation needed for recharging, effectively easing concerns of the adoption of EVs. One may expect that charge time can be easily predicted using the size of the battery and the speed of charging. Many [sources](#) cite this formula for calculating charge time that we will use as our baseline:

$$\text{Charge Time (hrs)} = (\text{Battery capacity (kWh)}) * (\text{Target SOC} - \text{Current SOC}) / (\text{Power (kW)})$$

The challenge with this formula is it makes a lot of simplifying assumptions that don't work well for our dataset. It assumes that charging speed is constant and known, the usable range of the battery is known, the battery has not degraded at all, and that there are no losses or temperature related effects. We plan to improve upon this calculation using regression and sequential machine learning models. The input to our algorithm is time series charging data, and we use linear regression, random forest, XGBoost, as well as RNNs with LSTM, GRU, TiDE, and TFT to predict the estimated time remaining charging.

**Related work:** Looking at prior research on this topic, the paper from Beijing on predicting charging time of battery electric vehicles (Bi, Wang, Sun, & Guan, 2018) utilized a combination of regression and time-series methods for prediction. The main strength associated with this paper's approach is the thorough collection of data from 70 battery electric vehicles over a long timeframe, with a dataset containing 41,400 charging processes. The central limitation that this paper acknowledged was that the study is based on data collected from exclusively Beijing-based battery electric vehicles, meaning there is a limit on the generalizability of the results to areas outside Beijing. It was additionally stated that potentially impactful external environmental factors such as charging station state and temperature were not considered in the model. The study's state-of-the-art-method was the integration of basic regression and time-series methods. This composite approach accounts for both the relationship between charging time and important features while addressing prediction errors from unobservable factors through the usage of time-series methods.

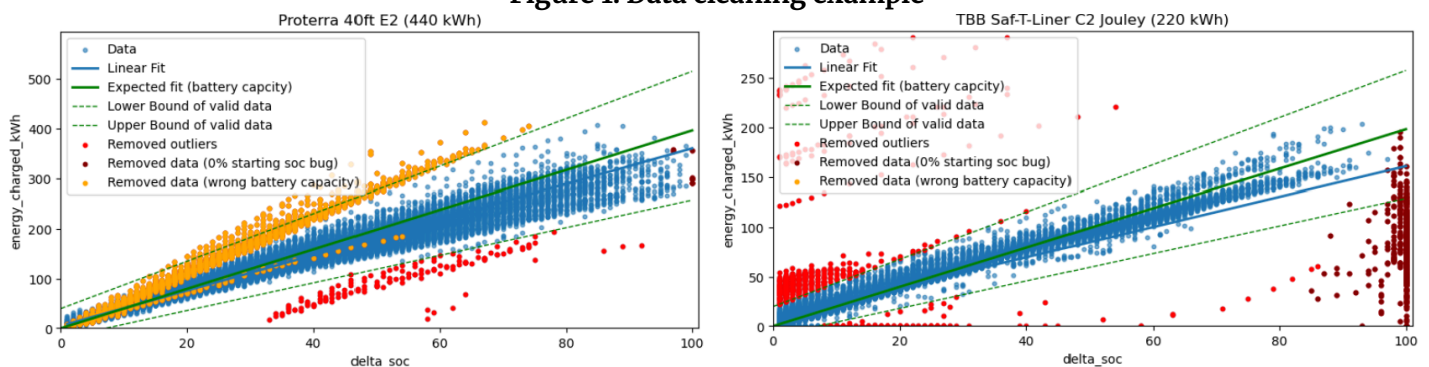
Another related paper, "Prediction of Electric Vehicle Charge Profile using Battery Digital Twin" (Rath & Wilkins, 2024), leveraged a digit twin (DT) framework that allows for continuous parameter updates through comparing predicted charging with the actual charging data. The key strength of this approach lies in its bi-directional data exchanges between physical and digital systems, allowing for accurate, real time alignment on prediction accuracy. The weaknesses that this paper encountered were that the model is less accurate at high or low ambient temperatures due to limited training data. The study's state-of-the-art-method was the DT framework's use of bi-directional data exchanges between physical and digital systems. This is effective because the framework supports optimization by consistently calibrating the model with sensor data from the battery management system.

**Dataset:** The data we are using comes from Camber's data lake, containing data that comes from 460 chargers across the US and 760 different vehicles that are 15 different models between January 2024 and July 2025. The vehicles range from transit buses, school buses, vans and cars. The chargers are privately owned and for fleets of EVs so the same vehicles are coming back repeatedly. The data is based on the open charge point protocol

(OCP), which is a common communication standard for EV chargers across vendors. Every minute that vehicles are charging, the following metrics are recorded for the charging session: power, energy, current, voltage, SOC, power offered (the power that the charger offers the vehicle), as well as an ID for the vehicle charging. There is additional human entered data on charger and vehicle specifications. The vehicle specifications include: make, model, year, battery capacity and in-service date (to get a sense of battery age). The relevant charger specifications include the current limitation on the cable of the charger. The team will start with pre-downsampled and pre-joined data from Camber's data platform. It is downsampled for the first time the SOC switches to reduce data size. The data has a column `unique_session_id` that defines a session based on a transaction ID and consecutive time with the same power offered. The outcome variable is the time remaining to get to the first data point where the session hits the final SOC, which we will refer to as the target SOC. Many vehicles stay plugged-in and charging slowly for hours after they have hit their final SOC. Weather data was supplemented into the dataset from [Visual Crossing](#). Using this weather data API, we added hourly temperature based on respective zip code and the datetime of each time-series point. The raw data was made up of 50,902 sessions and 1,574,957 datapoints, and after processing the data and adhering to our data cleaning procedures, 39,518 sessions, and 1,182,368 data points remained. We used 70/15/15 training, validation, and testing splits with a temporal split between training and validation/test within each vehicle group (type/make/model combo). This is in order to isolate the validation and training set from being behind any training set points. We split by each vehicle group so each is properly represented in the set. This process yielded 791,560 examples for training, 176,116 for validation, and 175,174 for testing.

**Data Preprocessing:** Given the large volume of data, we were able to eliminate data abnormalities without compromising our data split sizes. In order to ensure accurate predictive generalization, it was necessary for the model to be trained on high-quality charging behavior without data collection bugs, data quality issues, or examples of vehicle or charger malfunctioning. Additionally, we chose to only use a charging session with complete data. Some data processing was done in Camber's data platform to reduce volume of data, such as removing rows with missing values from unknown vehicle models. We removed sessions that had durations longer than 10 hours, sessions that ended at less than 85% SOC, sessions with SOC changes that are greater than 5%, and sessions with mid-session drops in SOC. For the purpose of adhering model training towards physically realistic data, we found and removed cases where there was an increasing SOC with zero input power, and ensured battery capacity was consistent with the total energy charged based on the percentage of the battery charged. **Figure 1**, shows the change in SOC vs the energy and the data points need to fall within a specified range around the line that represents battery capacity, with data points outside the boundaries being thrown out. This methodology showed that a subset of vehicles had inaccurate capacity measurements and put into question sessions that start at SOC = 0. We additionally dropped sessions where invalid zipcodes disallowed the addition of temperature data. Feature engineering was conducted deriving the days a vehicle has been in service, the energy and time change between timesteps of sequence data, and our target variable: time remaining. We scaled our data using a z-score on the training data mean and standard deviation, incorporating one-hot encoding of categorical fields such as vehicle make, model, and type.

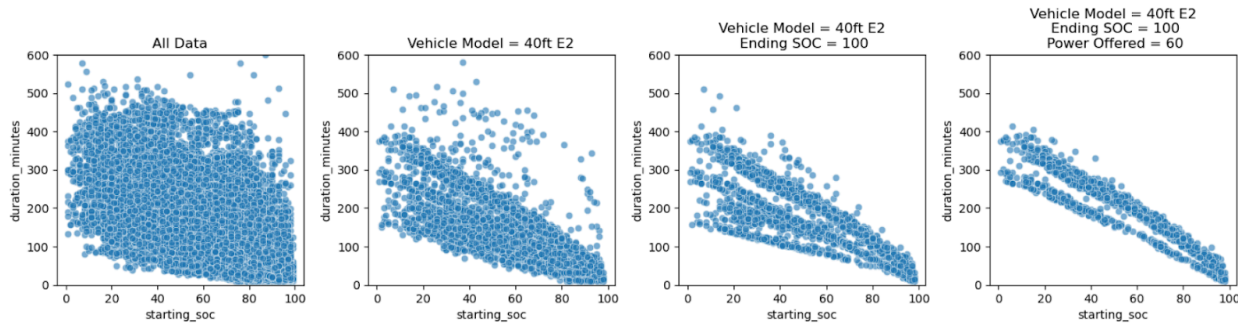
**Figure 1: Data cleaning example**



**Exploratory Data Analysis:** As part of EDA, we generated scatterplot matrices to visualize the relationships between charging session durations and all features. **Figure 2** shows the session duration against the current SOC, starting with all covariates and progressively adding more filters on other dimensions. When we filter for a specific vehicle model and battery capacity, we can start to see signs of a linear trend, and as we all filter for ending SOC and power offered, the trend becomes more defined. Interactions between each of these features will need to be taken into account by the model for these trends to be learned. The correlation matrix (see [appendix Figure A](#)) also shows that SOC-related variables are the highest correlated features with the target variable

compared to other features. Looking at the overall session, we found a time-based trend in the estimated battery size which was calculated by looking at the energy charge divided by the difference in SOC. By looking at vehicles with high volumes of data spread over multiple months, we found that the estimated battery size degraded around 2.5% over a year. The model is able to account for this by using the days in the service column.

**Figure 2: Exploratory Data Analysis (EDA)**



**Methods:** We determined the efficacy of a model by minimizing the MAE and root mean squared error (RMSE) of predicted time remaining on a test set, measured in minutes. The predictions are designed to be real-time with new predictions made intermittently during the session. Models were trained using each point in the time-series data and predicting the time remaining. We tried 8 different model architectures: linear regression, random forest, feedforward neural network, XGBoost, GRU, LSTM, TiDE, and TFT. The first 4 being regression models and the last 4 with sequence frameworks. The regression models use the current state features for each point along the session to make a prediction for the estimated time remaining. The LSTM and GRU are recurrent neural network frameworks (RNN), the Temporal Fusion Transformer (TFT) is a variation of a time-series centric transformer, and the Time-series Dense Encoder (TiDE) is a dense Neural Network oriented towards time-series. The time-series predictions for the LSTM and GRU were done with a 5 point sliding window where at each point the time remaining in the charge session is predicted. If there were not 5 points available, it was padded with zeros. Time-series predictions are done with a 5 datapoint sliding window for TiDE and 2 datapoint sliding window for TFT, both without padding.

For the linear model, we used Keras tensor flow to build a neural network without hidden layers. This is for a baseline comparison with other models and to understand the complexity of the problem. The random forest was built with sklearn RandomForestRegressor. We expected the feedforward neural network and XGBoost to perform the best out of the set of regression models we utilized. The neural network, built using TensorFlow, is well-suited to learn the linear and nonlinear part of EV charging. As shown in [Figure B](#) in the appendix, oftentimes the first part of the curve is linear, with a slope proportional to the power of the charger, and then becomes nonlinear in a way distinct to each vehicle. XGBoost decision trees are well-suited for data with multiple predictive breakpoints, which is the case with how low SOCs are faster to charge, while high SOCs are slower to charge. XGBoost is additionally compatible with feature importance analysis that can be used to detect data leakage and whether physical principles are properly used for predictions. XGBoost training and tuning is also highly parallelizable and relatively fast, allowing for thorough hyperparameter optimization. For all of the regression models, we use the following features: current SOC, target SOC, power offered (kW), current power import (kW), current voltage (V), temperature (F), battery capacity, vehicle make, vehicle model, vehicle year, vehicle type, days in service, and cable amperage.

LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) are specialized sequence models that can detect complex trends in input sequences and output scalars or additional sequences. LSTM models feature forget, input, and output gates to capture longer-term trends than typical sequence analysis models, while GRUs are architecturally simpler with just an update and reset gate that allow it to have a rudimentary understanding of when to keep and forget past information in a sequence. Both models are computationally inexpensive to tune and train compared to more specialized time series models. The LSTM and GRU were also built using the Keras TensorFlow sequential API and tuned with the Keras tuner. The sequences were created using the last 4 times that the SOC switched, as well as the current data point. Our timeseries features include: SOC, power offered (kW), power import (kW), voltage (V), current (A), and temperature (F). Since our data downsampled to the nearest time as the SOC switches, we also need to include a feature for the change in time between data points. The static vehicle parameters were duplicated 5 times for each time point.

The TFT (see [appendix Figure C](#)) and TiDE (see [appendix Figure D](#)) models were built using the Darts framework and both inputted with Darts Timeseries object sequences of past covariates, future covariates, and a

target sequence. The past covariates sequence contains the same power, vehicle, and environmental variables as the LSTM and GRU. The future covariates sequence contains the target SOC duplicated for the duration of the sequence, to indicate to the model that the known event occurring in the future timestep is the car charging to a certain SOC. The target sequence contains the elapsed time at each timestep, beginning at 0 at the start of the input window, with the elapsed time of the final, masked, timestep corresponding with the highest SOC being the prediction scalar. To prevent data leakage due to non-final target sequence values being visible to the model, this model predicts 'elapsed time corresponding to final SOC' instead of 'time remaining until final SOC', which is functionally equivalent. Both TFT and TiDE are given identical input series but tuned utilizing Bayesian optimization with distinct hyperparameters.

**Experiments, Results and Discussion:** We split up our results into cold prediction and prediction after SOC has increased once due to the TFT and TiDE requiring multiple data points as input. A cold prediction, or cold start, is a prediction without any sequence data available. The regression models are a natural fit for this since they do not use any sequence data. The LSTM and GRU are able to utilize array padding to make cold start predictions, while the TFT and TiDE models use Darts Timeseries sequences that are incompatible with padding schemes. We separated the cold predictions to compare all models for non-cold predictions, as shown in **Table 1**. All results refer to the non-cold predictions to make comparisons on performance to the TiDE and TFT models, unless specified otherwise.

We calculated the model performance on the baseline calculation described in the introduction section. Since our data was split temporally, there are some differences between the train and test data. For example, the MAE for the train was 5 minutes higher. The baseline test MAE is 57 minutes, which is an unacceptable error considering the average time remaining is 100 minutes. The linear model improved upon the baseline by 17 minutes, but the resulting model yielded predictions of negative time remaining due to the nonlinearity of the data. The random forest performed better than the linear model with an 18 minute improvement on MAE, but was quick to overfit. To account for this, we used sklearn's RandomizedSearchCV and discarded any models where the MAE was 10 less for the train than the test for both the random forest and XGBoost. **Table A** in the appendix shows selected hyperparameters for random forest alongside all other models. The results of XGBoost proved to be the best for the regression models that excluded sequence data, with an MAE of 15.76 minutes.

The feedforward neural network was built with Keras TensorFlow and trained with the Keras tuner with up to 4 layers and drop out and regularization between each layer. Regularization was held constant at each layer, and the units and dropout was cut in half at each layer to reduce tuning space. The best performing model was a 4 layer model with 96 neurons, 0.3 dropout, and 0.0001 regularization on the first layer. ReduceLROnPlateau was used to reduce the learning rate upon validation MAE plateauing. The neural network outperformed the random forest by 5 minutes MAE, but did not perform as well as XGBoost, albeit with less overfitting than XGBoost. For the LSTM and GRU, we used the Keras TensorFlow hyperparameter tuner to let the model have up to 3 LSTM/GRU layers and up to 3 dense layers. Layer size was halved with each subsequent layer to reduce the number of hyperparameters. Decreasing dropout frequencies were utilized between each layer. The best model for the LSTM featured 3 LSTM layers and 3 dense layers while the GRU converged upon a simpler model with 2 GRU layers and 1 dense layer. Both models preferred low drop out with the first layer being 0.1. The GRU outperformed the LSTM at 3% of the parameter count with an MAE of 12.8 and 13.29 minutes respectively. However, the LSTM did have a lower mean absolute percentage error (MAPE), suggesting that it performs better on shorter durations.

TFT and TiDE models were created using the Darts library and packages the time series data into sequences of past covariates, future covariates, and a target sequence that they attempt to predict. The TFT and TiDE models are generally used for multivariate short-term next-step predictions in weather/climate data, so inverting the architecture to predict scalars leaves a lot of predictive power on the table. The TiDE model was very quick to overfit on the data, while the TFT model was extremely slow to train and difficult to tune but tended to have good generalization with low overfitting. The TiDE model showed the lowest overall MAPE at 13.17%.

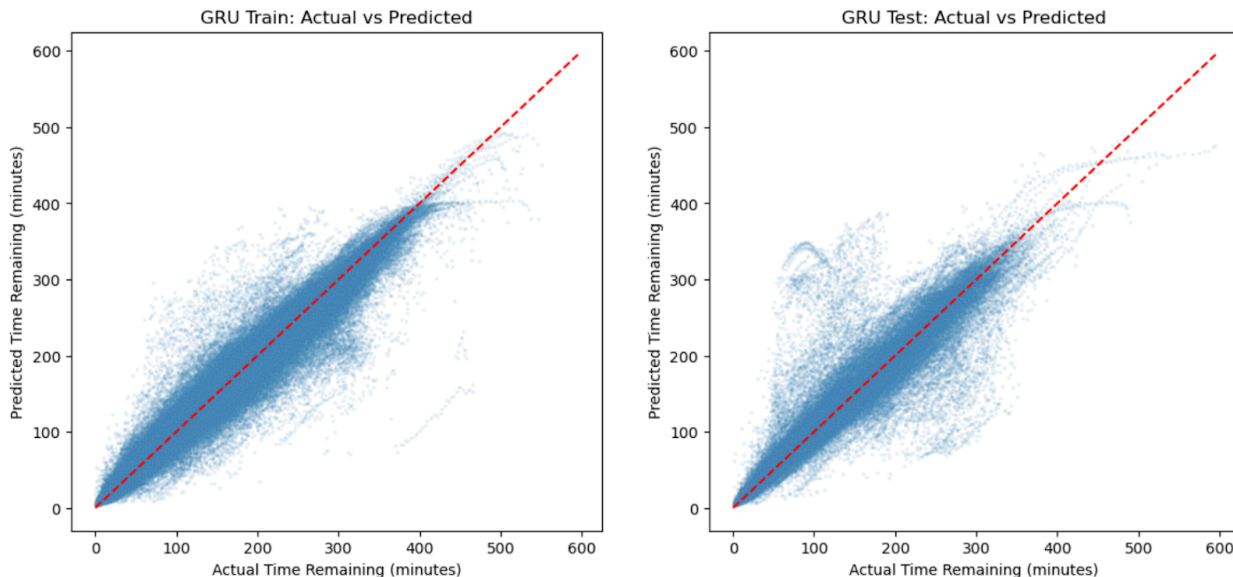


**Table 1: Model results**

|                    | Parameters | Cold Prediction (first session point only) |       |       |       | Prediction (After first point) |       |       |       |         |
|--------------------|------------|--------------------------------------------|-------|-------|-------|--------------------------------|-------|-------|-------|---------|
|                    |            | Train                                      |       | Test  |       | Train                          |       | Test  |       |         |
| MAE/RMSE (Minutes) |            | MAE                                        | RMSE  | MAE   | RMSE  | MAE                            | RMSE  | MAE   | RMSE  | MAPE    |
| Baseline           | -          | 56.71                                      | 84.81 | 54.74 | 82.11 | 62.16                          | 90.69 | 57.26 | 84.59 | 46.89 % |
| Linear Model       | 39         | 40.07                                      | 52.85 | 41.09 | 54.60 | 37.38                          | 50.16 | 40.17 | 52.50 | 74.28%  |
| Random Forest      | -          | 18.61                                      | 28.25 | 24.29 | 37.34 | 15.21                          | 23.45 | 21.91 | 32.89 | 31.41%  |
| XGBoost            | 1.3M       | 10.05                                      | 18.48 | 20.91 | 37.9  | 7.20                           | 12.86 | 15.76 | 30.01 | 14.64%  |
| Neural network     | 10k        | 17.99                                      | 30.36 | 21.57 | 36.21 | 11.6                           | 20.75 | 16.13 | 28.71 | 17.08%  |
| LSTM               | 2.1M       | 16.79                                      | 28.14 | 23.81 | 40.96 | 8.43                           | 14.37 | 13.29 | 26.70 | 13.91%  |
| GRU                | 63K        | 15.50                                      | 25.74 | 23.00 | 42.26 | 7.90                           | 13.66 | 12.80 | 26.03 | 15.00%  |
| TiDE               | 83K        | -                                          | -     | -     | -     | 10.50                          | 20.50 | 17.32 | 35.34 | 13.17%  |
| TFT                | 465K       | -                                          | -     | -     | -     | 12.80                          | 24.30 | 18.46 | 40.70 | 15.69%  |

Looking at MAE and RMSE our best performing model was the GRU, but looking at MAPE, the TiDE model was the best. This may mean that the GRU is more well suited for longer-term prediction consistency with its lower MAE and RMSE, while the TiDE model is more consistent with correctly predicting shorter charge sessions. Our objective was to minimize MAE and RMSE so we chose the GRU as our best performing model. For the cold predictions the XGBoost model did the best, but the GRU did well enough with a much smaller parameter size that, if deployed in industry, would allow for it to sufficiently cover both predictive classes.

Our GRU was a comparatively simple model, having only 63k parameters compared to our 2.1M parameter LSTM and our 465K TFT. **Figure 7** shows the true vs predicted results for both train and test, with the diagonal line representing a perfect prediction, and distance from the line indicating predictive error. There are differences between the train and test performance with the number of points further away from the diagonal line, however we do not believe this model is overfitting as performance is not fundamentally different between training and testing sets.

**Figure 7: GRU model results**

We also performed a sub group analysis by grouping every feature, including continuous variables, and ranking them by MAE and MAPE. **Table 3** shows the top 10 worst subgroups by MAE and MAPE. When it comes to MAE, the model does worse with low starting SOC and sessions with overall a long time remaining. It includes vehicles with current SOC of 0% to 30%, voltages that correspond to low SOC, and the vehicle type with the biggest capacity in the dataset, the Proterra ZX5 Max. These results are expected because the longer your time remaining is, the larger error we would expect. Additionally, evaluated the subgroups with the worst MAPE. This primarily included under represented vehicles models in our dataset. The vehicles with the worst MAPE are the RAM Promaster and Karsan E-Jest with 739 and 4,768 data points respectively. This compared to the 318,669 data points for the TBB Saf-T-liner Jouley school bus meaning that the model didn't focus a lot on learning these 2 vehicle models. Future work could be done to duplicate or supplement data for these vehicle types to improve performance. Additionally, the model had a worse performance for the very high temperatures, above 95°F, when it comes to MAPE. This also was an underrepresented group in the dataset with only 3,319 data points.

**Table 3: GRU Sub group analysis top 10 worst ranked by Validation MAE and MAPE**

| Sub group ranked by worst MAE           |         |             | Sub group ranked by worst MAPE       |          |             |
|-----------------------------------------|---------|-------------|--------------------------------------|----------|-------------|
| Feature group                           | Val MAE | Train Count | Feature group                        | Val MAPE | Train Count |
| Current voltage: (0, 330]               | 32.83   | 7,282       | Vehicle make: RAM                    | 0.40     | 739         |
| Current SOC: (0, 10]                    | 27.04   | 547         | Vehicle model: Promaster             | 0.40     | 739         |
| Current voltage: (570, 600]             | 26.31   | 406         | Battery capacity kWh: (100.0, 200.0] | 0.40     | 739         |
| Vehicle year: 2025                      | 26.18   | 2,938       | Current voltage: (390, 420]          | 0.33     | 10,182      |
| Current SOC: (10, 20]                   | 26.17   | 3,608       | Temperature F: 95+                   | 0.31     | 3,319       |
| Current Power Import kW: (180.0, 200.0] | 25.72   | 161         | Vehicle make: Karsan                 | 0.26     | 4,768       |
| Current Power Import kW: (160.0, 180.0] | 25.47   | 746         | Vehicle model: E-Jest                | 0.26     | 4,768       |
| Current SOC: (20, 30]                   | 22.06   | 10,918      | Vehicle type: Passenger Car          | 0.21     | 10,500      |
| Target SOC: 99.0                        | 21.87   | 230,050     | Battery capacity kWh: (0.0, 100.0]   | 0.21     | 16,941      |
| Vehicle model: 40ft ZX5 Max             | 20.28   | 26,325      | Vehicle make: Volkswagen             | 0.20     | 9,761       |

With this problem being based in physics, there are certain features that should be considered more important than others for the model to be widely generalizable. It should be the case that the model inherently understands the baseline physics calculation, and augments it with specific car and environmental data. For example, most school buses in our dataset are charged at 60kW chargers so the model could get away without learning how different chargers would charge those vehicles. One way we can see if the models learned the physics is by looking at feature importance. **Table 4** shows the top 10 most important features for the linear model, random forest, XGBoost and neural network. Feature importance for the linear model is determined by model weights. For random forest and XGBoost, there is built in feature importance to the model and for the neural network we use the SHAP library. Based on the baseline calculation the most important features should be: battery capacity, current SOC, target SOC, and power offered or power import. Looking at the top 10 most important features we

can see that at least 3 out of the 4 are included. The linear model struggles to learn the importance of battery capacity and instead uses the vehicle models. The random forest and XGBoost model include the 4 most important features in the top 10, but XGBoost weighs one of the vehicle models the most important feature. The neural network does well with the top 4 features (voltage is highly correlated with current SOC), but struggles to see the importance of power. The full SHAP plot for the neural network is included in **Figure E** in the appendix.

**Table 4: Feature Importance (Non-time series models)**

|                                       | Linear Model<br>(feature weights)                                                                                                                                                                                                                                                                                             | Random Forest<br>(built-in)                                                                                                                                                                                                                                                                                           | XGBoost<br>(built-in)                                                                                                                                                                                                                                                                                          | Neural Network<br>(SHAP)                                                                                                                                                                                                                                                                                                   |
|---------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Top 10 most important features</b> | <ol style="list-style-type: none"> <li>1. Target SOC</li> <li>2. Current SOC</li> <li>3. Vehicle Year</li> <li>4. Current Voltage</li> <li>5. Model 35ft XR+</li> <li>6. Power Offered</li> <li>7. Model Xcelsior CHARGE NG</li> <li>8. Make New Flyer</li> <li>9. Model 35ft ZX5+</li> <li>10. Model 40ft ZX5 Max</li> </ol> | <ol style="list-style-type: none"> <li>1. Target SOC</li> <li>2. Current SOC</li> <li>3. Current Voltage</li> <li>4. Battery Capacity</li> <li>5. Days in service</li> <li>6. Power Offered</li> <li>7. Vehicle year</li> <li>8. Temperature</li> <li>9. Make TBB</li> <li>10. Model Saf-T-Liner C2 Jouley</li> </ol> | <ol style="list-style-type: none"> <li>1. Model 35ft ZX5+</li> <li>2. Battery capacity</li> <li>3. Target SOC</li> <li>4. Vehicle year</li> <li>5. Model 40ft E2 Max</li> <li>6. Current SOC</li> <li>7. Model 35 ft XR+</li> <li>8. Make TBB</li> <li>9. Power Offered</li> <li>10. Cable amperage</li> </ol> | <ol style="list-style-type: none"> <li>1. Current Voltage</li> <li>2. Battery Capacity</li> <li>3. Target SOC</li> <li>4. Current SOC</li> <li>5. Make TBB</li> <li>6. Make Proterra</li> <li>7. Type Transit Bus</li> <li>8. Model 40ft ZX5+</li> <li>9. Model Saf-T-Liner Jouley</li> <li>10. Type School bus</li> </ol> |

**Conclusions:** The combination of low parameter count and superior sequence prediction results for the GRU signal that the architecture of the GRU is suited to this problem without requiring a large hypothesis space that would be enabled by more parameters. XGBoost was tested to be marginally better at cold predictions, but at the cost of over 20 times more parameters and significantly higher comparative overfitting. Therefore, in a production deployment of predictive models where performance would be relevant, we would recommend the use of the GRU for both classes of predictions.

One area for future improvement is on vehicle models and combinations of vehicles and chargers that the model has seen little to none of. One way to address this is with a mixed-methods approach with a combination of physics and machine learning. This could be implemented by incorporating the physics into the loss function or augmenting the data with a physics charge curve with a continuous spread of power values and battery capacities. Additionally, the sequential data models would benefit from data that wasn't downsampled and additional computing resources to train on this high volume of data and large parameter models.

## Contributions

**Maggie** has been key in acquiring and refining our data, since she has direct access to the database at her workplace. She's been writing the queries to pull new data and constantly looks for ways to enhance the dataset with whatever additional information she can access. Maggie spearheaded keeping the models apples to apples between the regression and sequential models including standardizing the data processing. She worked on the linear model, random forest, XGBoost, NN, LSTM and GRU.

**Alex** has been focused on enhancing our feature set by bringing in environmental data from external sources, guided by their importance in existing literature. In addition, explored models including linear regression, random forest, and neural network.

**Alexandra** explored both a gradient boosted trees model and a recurrent neural network with LSTM layers.

**Micah** has worked on data pipelines and implementation of TFT, TiDE, and LSTM modeling techniques the time-series data, as well as focusing on time-series data cleaning, allowing for a results-centric method of feature selection and outlier detection. He architected the approach of using a recurrent neural network for the time-series data and led hyperparameter/model tuning. He has worked on GPU acceleration of model training and other optimizations.



## References

- Bi, J., Wang, Y., Sun, S., & Guan, W. (2018). Predicting Charging Time of Battery Electric Vehicles Based on Regression and Time-Series Methods: A Case Study of Beijing. *Energies*, 11(5), 1040.  
<https://doi.org/10.3390/en11051040>
- Rath, Subhajeet & Wilkins, Steven. (2024). [Prediction of Electric Vehicle Charge Profile using Battery Digital Twin](#). 10.13140/RG.2.2.28145.08800.
- Shi, J., Tian, M., Han, S., Wu, T.Y. and Tang, Y., 2022. [Electric vehicle battery remaining charging time estimation considering charging accuracy and charging profile prediction](#). *Journal of Energy Storage*, 49, p.104132.
- EV Charging Time Calculator: Estimate Your Charging Duration. (n.d.). [Www.chargie.com](https://www.chargie.com/resources/how-to-estimate-your-evs-charging-time).  
<https://www.chargie.com/resources/how-to-estimate-your-evs-charging-time>
- Hyundai Ioniq 5 Long Range 2WD charging curve & performance | EVKX.net. (2025). [Evkx.net](https://evkx.net/models/hyundai/ioniq_5/ioniq_5_long_range_2wd/chargingcurve/).  
[https://evkx.net/models/hyundai/ioniq\\_5/ioniq\\_5\\_long\\_range\\_2wd/chargingcurve/](https://evkx.net/models/hyundai/ioniq_5/ioniq_5_long_range_2wd/chargingcurve/)
- Efficient EV Charger Management Software | Camber Core. (2025, May 20). *Camber Charging | Reliable EV Fleets*.  
<https://cambercharging.com/camber-core-ev-fleet-charge-management-software/>
- Weather Data & Weather API | Visual Crossing. (n.d.). [Www.visualcrossing.com](https://www.visualcrossing.com/). <https://www.visualcrossing.com/>

Appendix

Figure A: Correlation Matrix

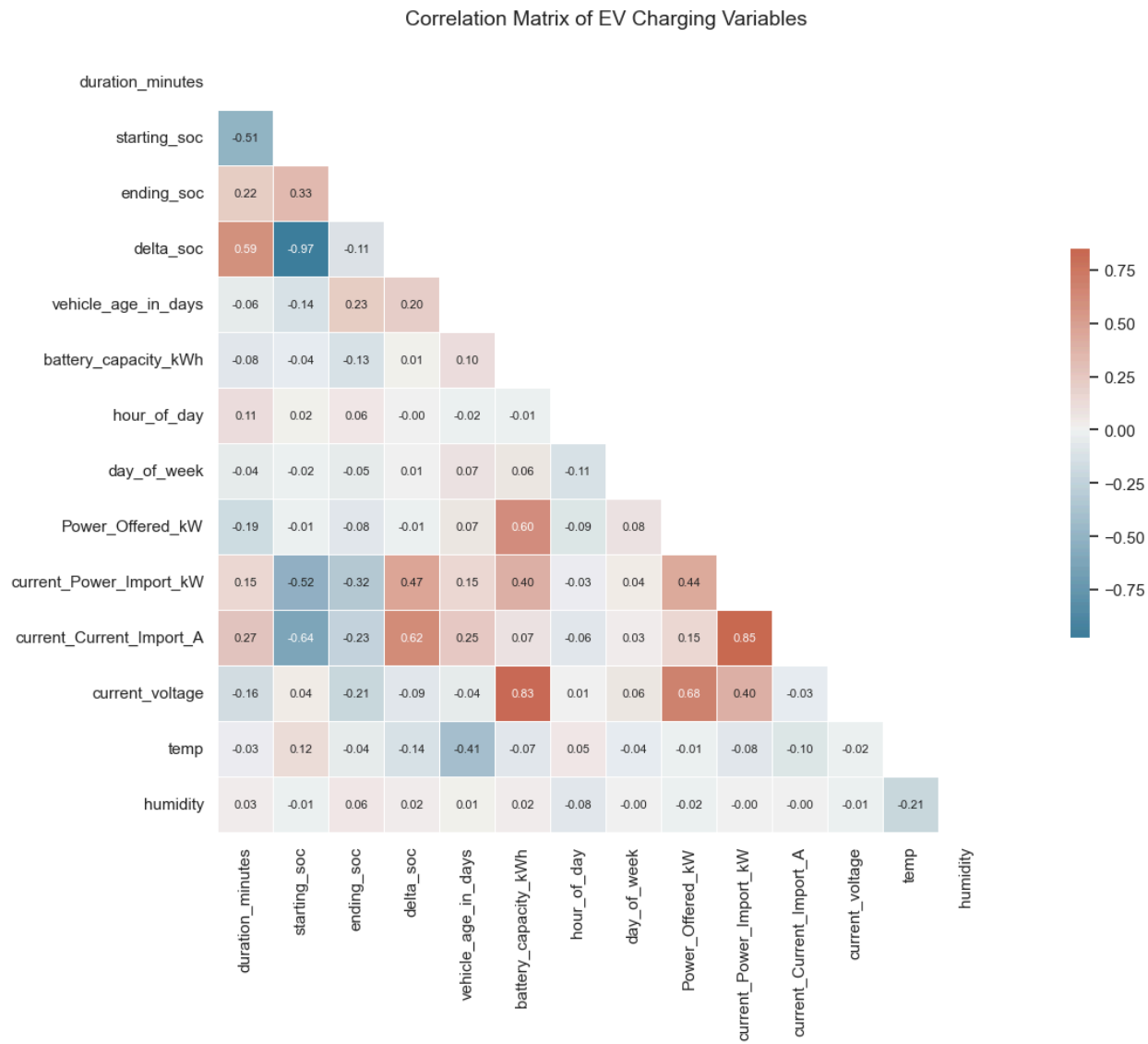


Figure B: Charging Curves

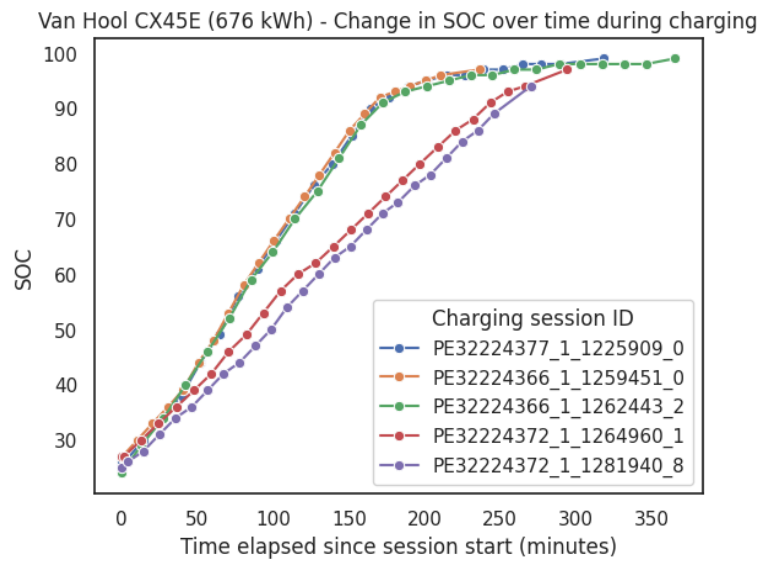


Figure C: TFT Modeling Architecture

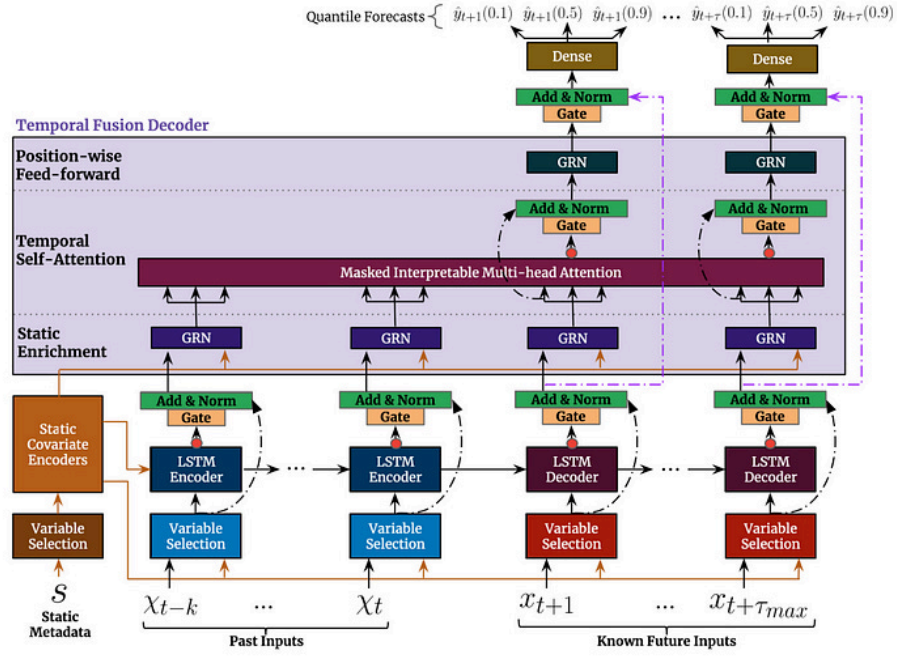
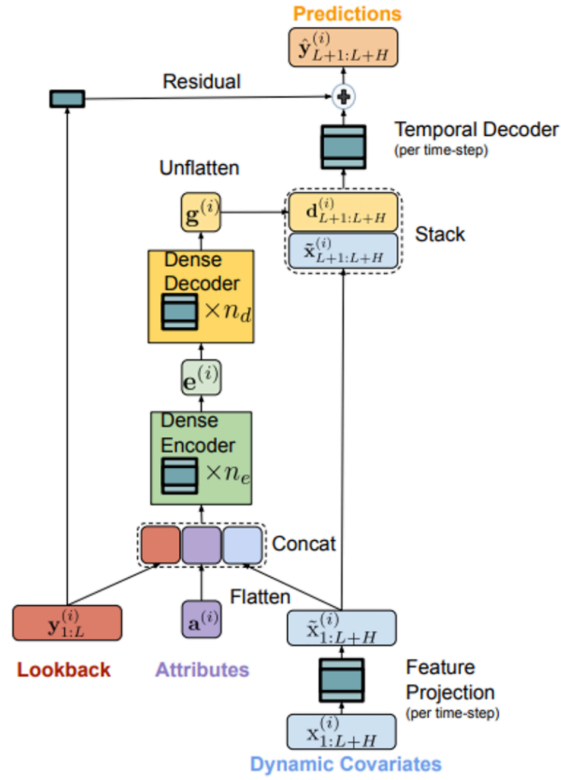


Figure D: TiDE Modeling Architecture



**Table A: Models selected hyper parameters**

| <b>Model</b>          | <b>Hyper parameters</b>                                                                                                                                                                                                                                                                                                  |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Linear Model</b>   | learning_rate = 0.01                                                                                                                                                                                                                                                                                                     |
| <b>Random Forest</b>  | n_estimators: 100<br>min_samples_split: 5<br>min_samples_leaf: 2<br>max_features: sqrt<br>max_depth: 15                                                                                                                                                                                                                  |
| <b>XGBoost</b>        | Subsample: 0.9<br>Reg lambda: 0.2<br>Reg alpha: 0.4<br>Num estimators: 150<br>Max depth: 12<br>Learning rate: 0.05<br>Colsample by tree: 1                                                                                                                                                                               |
| <b>Neural Network</b> | num_layers: 4<br>1st layer units: 96<br>2nd layer units: 48<br>3rd layer units: 24<br>4th layer units: 12<br>1st layer dropout: 0.3<br>2nd layer dropout: 0.15<br>3rd layer dropout: 0.075<br>4th layer dropout: 0.0375<br>activation: tanh<br>regularizers: 0.001<br>learning_rate: 0.001 (with ReduceLROnPlateau)      |
| <b>LSTM</b>           | LSTM layers: 3<br>Dense layers: 3<br>LSTM units layer 1: 512<br>LSTM units layer 2: 256<br>LSTM units layer 3: 128<br>Dense units layer 1: 128<br>Dense units layer 2: 64<br>Dense units layer 3: 32<br>Drop out between all layers: 0.1, 0.07, 0.05, 0.06, 0.03, 0.02<br>Learning rate: 0.0001 (with ReduceLROnPlateau) |
| <b>GRU</b>            | GRU layers: 2<br>Dense layers: 1<br>GRU units layer 1: 96<br>GRU units layer 2: 48<br>Dense units layer 1: 32<br>Drop out between all layers: 0.1, 0.07, 0.06<br>Learning rate: 0.0001 (with ReduceLROnPlateau)                                                                                                          |
| <b>TiDE</b>           | input_chunk_length: 5<br>output_chunk_length: 1<br>num_encoder_layers: 3<br>num_decoder_layers: 3<br>hidden_size: 70                                                                                                                                                                                                     |

|            |                                                                                                                                                                                                                                                                                                                                                                              |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | temporal_decoder_hidden: 35<br>dropout: 0.2<br>batch_size: 512<br>epochs: 300<br>loss_function: Val MAE<br>optimizer: RAdam<br>learning_rate: 0.006<br>weight_Decay: 0.0002<br>learning_rate_decay: learning_rate * 0.965 per epoch                                                                                                                                          |
| <b>TFT</b> | input_chunk_length: 2<br>output_chunk_length: 1<br>lstm_layers: 3<br>dropout: 0.05<br>batch_size: 2048<br>hidden_continuous_size: 4<br>num_attention_heads: 4<br>hidden_size: 64<br>epochs: 200<br>loss_function: Val MAE<br>optimizer: AdamW<br>feed_forward: SwiGLU<br>learning_rate: 0.005<br>weight_Decay: 0.0001<br>learning_rate_decay: learning_rate * 0.96 per epoch |

Figure E: SHAP feature importance (Neural Network)

